

Zero-Sum Deletion Normal Forms for Multi-Tag Quantum Compilation

Sze Chun Yiu
sze-chun.yiu@fysik.su.se

25 August 2026

Abstract

Finite-support normal forms make exact quantum-compiler optimization enumerable, but a rank count alone does not establish that a deletion is semantically sound or cost-nonincreasing. Let H be a finite abelian signature group and $A \subseteq H$ an allowed alphabet. Write $\text{zsf}(H; A)$ for the largest length of a word over A with no nonempty zero-sum subsequence, where a subsequence is selected by an arbitrary set of positions and need not be contiguous. We prove that every finite instance of a deletion-closed compiler grammar has an exact optimum of support at most $\text{zsf}(H; A)$, provided each constrained generator has nonzero total signature and every zero-signature deletion preserves feasibility without increasing cost. For a binary group generated by its realized alphabet, $H = \langle A \rangle \cong \mathbb{F}_2^d$, elementary linear algebra gives $\text{zsf}(H; A) = d$.

We instantiate the theorem in an arbitrary-block MultiTag Tag-and-Restore Encoding (TARE) grammar. With $b \geq 2$ ordered blocks, $s \geq 0$ shared Tags, minimum frame coefficient μ , and Restore coefficient t_R , changing one local letter increases the b -way Restore functional by at most $b - 1$, and this bound is sharp. Consequently, throughout the sufficient cone $\mu \geq (b - 1)t_R$, every instance has an optimum satisfying $|\text{supp}(R)| \leq \text{zsf}(H_R; A_R) = \text{rank}(H_R) \leq s + 1$ for every frame R . We make no intrinsic necessity claim for any specialization or outside the sufficient cone. The result separates the finite-group certificate, the compiler semantics that make deletion sound, and the objective inequality that makes deletion profitable.

Keywords: quantum compilation; block encoding; exact normal forms; zero-sum sequences; sparse optimization

1. Introduction

Tag-and-Restore Encoding (TARE) is an upstream block-encoding construction that exposes frame, Tag, branch, and Restore choices. The present work studies exact normal forms for those auxiliary compiler choices; it does not claim the underlying block-encoding primitive [1].

A standard support proof attaches a finite binary syndrome to every active coordinate. When support exceeds the syndrome dimension, linear dependence produces a removable zero-syndrome subset. The dimension obtained in this way can look like an intrinsic property of the compiler, although the proof uses only two ingredients: a sufficiently long signature word contains a zero-sum subsequence, and deleting that subsequence is non-increasing. The primitive threshold is therefore a zero-sum threshold, while its compiler use still depends on semantic and objective proofs.

This distinction matters for three reasons. First, it extends the deletion argument from binary vector spaces to finite abelian signature groups. Second, in the binary specialization it identifies realized

rank rather than an arbitrary ambient dimension. Third, it keeps the combinatorial certificate separate from the objective inequality that licenses the corresponding edit.

Our contributions are:

1. an alphabet-Davenport deletion theorem for deletion-closed optimization grammars with persistent soundness and dominance assumptions;
2. the binary rank corollary, including exactness for the deletion language when a basis word is allowed;
3. an arbitrary-block MultiTag-TARE instantiation with exact local Restore sensitivity $b - 1$ and sufficient cone $\mu \geq (b - 1)t_R$;
4. a one-Tag, three-block finite record illustrating the support-two ceiling without promoting it to an intrinsic theorem; and
5. an explicit boundary between certificate support, intrinsic support, and physical resource claims.

2. An alphabet-restricted zero-sum invariant

Let H be a finite abelian group written additively, and let $A \subseteq H$ be finite. A *subsequence* of a word is obtained by selecting an arbitrary set of positions, without any contiguity requirement. A word over A is *zero-sum-free* if none of its nonempty subsequences has sum zero. Define

$$\text{zsf}(H; A) = \max\{|W| : W \text{ is a zero-sum-free word over } A\}.$$

Words permit repetition, matching the multiplicity of compiler coordinates. For $A = H \setminus \{0\}$, the invariant equals $D(H) - 1$, where $D(H)$ is the classical Davenport constant. Smaller alphabets can yield strictly smaller values. We use explicit notation to avoid conflating this quantity with other weighted or restricted zero-sum conventions.

3. The deletion theorem

Fix an optimization grammar whose feasible set is finite and nonempty, so its objective attains a minimum. For each constrained generator R , every active coordinate q carries a signature $v_q \in A_R \subseteq H_R$. Assume the following conditions hold initially and after every admitted deletion:

1. **Nonzero total.** Every feasible constrained generator satisfies $\sum_q v_q \neq 0$.
2. **Deletion closure and soundness.** Replacing R by the identity on any nonempty zero-sum subsequence remains inside the admitted grammar and preserves every semantic constraint represented by the signatures.
3. **Persistent deletion dominance.** The same operation does not increase the objective.

Theorem 1 (alphabet-Davenport normal form). Every admitted finite instance has an exact optimum for which

$$|\text{supp}(R)| \leq \text{zsf}(H_R; A_R)$$

for every constrained generator R .

Proof. Begin with an optimum. If the signature word of R is longer than $\text{zsf}(H_R; A_R)$, it contains a nonempty zero-sum subsequence. The nonzero total prevents that subsequence from being the whole word. Delete its coordinates. Closure and soundness preserve feasibility, while dominance preserves optimality. Support strictly decreases. Because all three assumptions persist, the process terminates with support at most $\text{zsf}(H_R; A_R)$. Repeating the argument for each constrained generator is not interpreted as a single pass. Instead, while any frame violates its current bound, choose one such frame, recompute its current signature alphabet, and delete a current zero-sum subsequence. Every step strictly decreases total frame support, so this global descent terminates with all current bounds satisfied simultaneously. $\square$

The abstract statement localizes the compiler-specific burden: one must define the signature map, prove nonzero total and deletion closure, and account for the objective change. The zero-sum threshold alone supplies none of those facts.

4. Binary rank as a corollary

Let $H = \mathbb{F}_2^d$. Every word longer than d is linearly dependent and therefore contains a nonempty zero-XOR subsequence. Hence

$$\text{zsf}(\mathbb{F}_2^d; A) \leq d$$

for every $A \subseteq \mathbb{F}_2^d$. If A contains a basis, the basis word is zero-sum-free, so equality holds. In particular, if $H = \langle A \rangle$, then the spanning set A contains a basis of H , and equality is automatic.

Corollary 2. A rank- d deletion bound in an elementary binary signature system is exact for the stated deletion language when a basis word is allowed. It becomes an intrinsic compiler lower bound only when an independent compiler witness proves that support below d is impossible.

5. MultiTag-TARE grammar

Fix $b \geq 2$ ordered blocks. Block j contains two nonidentity target Pauli strings P_{j0}, P_{j1} , an ordered anticommuting frame pair (R_{j0}, R_{j1}) , a target permutation π_j , and one designated central case. There are $s \geq 0$ shared Tag Paulis $S_1, \dots, S_s$. For every Tag, the two case labels are prescribed to be opposite within a block and common across blocks. Target assignments, central cases, and Tags are held fixed during the deletion below.

For a frame $R = R_{jk}$, let $R' = R_{j,1-k}$. At every active coordinate, define

$$v_q = (\langle R_q, R'_q \rangle, \langle S_{1q}, R_q \rangle, \dots, \langle S_{sq}, R_q \rangle) \in \mathbb{F}_2^{s+1},$$

where $\langle \cdot, \cdot \rangle$ is the local symplectic commutation form. The XOR of the first components is one, so the total signature is nonzero.

The Restore strings are $T_{jk} = P_{j, \pi_j(k)} R_{jk}$, with Pauli products taken up to phase. Let $m_{jk} \geq \mu$ be the frame multiplier and let $u_\ell \geq 0$ be the Tag multiplier. Up to constants independent of the compiler choices, the complete structural objective used here is

$$C = \sum_{j,k} m_{jk} w(R_{jk}) + \sum_{\ell=1}^s u_{\ell} w(S_{\ell}) + t_R \sum_{q,k} F_b((T_{1k})_q, \dots, (T_{bk})_q),$$

where $t_R \geq 0$, w is Pauli support, and

$$F_b(a_1, \dots, a_b) = \begin{cases} 1, & a_1 = \dots = a_b \neq I, \\ |\{i : a_i \neq I\}|, & \text{otherwise.} \end{cases}$$

defines the local b -block Restore functional. This finite grammar has no additional feasibility predicate that depends on the local letters of a single frame beyond frame nonidentity, partner anticommutation, and the declared Tag labels. This sentence is part of the model definition, not an assertion about arbitrary block-encoding compilers.

6. Exact local Restore sensitivity

Lemma 3. Replacing one argument of F_b increases its value by at most $b - 1$, and the bound is attained.

Proof. Away from the all-equal nonidentity state, F_b is the ordinary nonidentity Hamming weight, so one replacement raises it by at most one. Entering the exceptional state lowers the value to one. Leaving that state while keeping the changed letter nonidentity changes the value from one to b , giving the exact increase $b - 1$. $\square$

7. MultiTag normal form

Let A_R be the alphabet actually realized by the partner/Tag signatures of frame R , and let $H_R = \langle A_R \rangle$.

Theorem 4. If $\mu \geq (b - 1)t_R$, every admitted MultiTag-TARE instance has an exact optimum satisfying

$$|\text{supp}(R)| \leq \text{zsf}(H_R; A_R) = \text{rank}(H_R) \leq s + 1$$

for every frame R .

Proof. A zero-signature subsequence preserves partner anticommutation and every Tag label; the Tags remain fixed. Deleting each selected coordinate refunds at least μ in frame cost and can add at most $(b - 1)t_R$ in Restore cost by Lemma 3. The deletion is therefore non-increasing in the stated cone. Its effect stays within the same grammar, and the signature invariants persist, so Theorem 1 applies iteratively. Finally, H_R is an elementary binary subgroup of $\mathbb{F}_2^{s+1}$, which gives the rank inequalities. $\square$

The equality in the middle is specific to the binary group generated by the realized alphabet. The general finite-abelian theorem can still improve the unrestricted Davenport ceiling, but no strict alphabet-versus-realized-rank refinement is claimed for this binary MultiTag specialization.

8. One-Tag, three-block finite record

For the one-Tag, three-block grammar,

$$b = 3, \quad s = 1, \quad \mu = 2, \quad t_R = 1.$$

The objective lies on the sufficient-cone boundary. The signature alphabet realizes a basis of $\mathbb{F}_2^2$, so the deletion certificate ceiling is two. An internal exact $n = 2$ record reports unrestricted optimum 5 and support-at-most-one optimum 6 over 13,824 feasible configurations. The public source package currently supplies the result record but not a standalone enumerator; accordingly, this finite observation is not used here to claim an intrinsic lower bound.

9. Relation to prior work

Sparse optimal solutions [2], Davenport constants, restricted zero-sum invariants [3], finite-field dependence, Pauli symplectic algebra, and exact synthesis are established areas. In particular, general sparse integer optimization provides objective-independent support bounds, while zero-sum theory provides sequence thresholds for selected alphabets and weights. Those results own the generic ideas of sparse optima and zero-sum thresholds.

The residual contribution here is their compiler-specific conjunction: a TARE-derived semantic signature, exact arbitrary-block Restore sensitivity, a sufficient objective cone, an alphabet-sensitive support normal form, and a separate one-Tag, three-block finite record.

10. Reproducibility and limitations

Finite-group controls, binary basis obstructions, and the exact local Restore sensitivity have been checked independently for bounded instances. The proofs above carry the all-size authority.

The theorem applies only when zero-sum deletion is grammar-closed, semantics-preserving, and non-increasing after every prior deletion. Computing $\text{zsf}(H; A)$ may itself be difficult. The MultiTag result applies only to the explicit grammar and objective. Outside $\mu \geq (b - 1)t_R$, the proof is silent; it does not imply that larger support is necessary. General multi-Tag sharpness remains open. Structural support is not physical T count, runtime, circuit depth, qubit count, or quantum advantage.

Data and code availability

The source package accompanying this manuscript contains finite-group and Restore-sensitivity records together with the one-Tag, three-block result record. The displayed arguments carry the all-size upper claim; no intrinsic lower claim depends on the packaged record. These files are distributed in the source archive accompanying this version.

11. Conclusion

The primitive deletion threshold is the longest zero-sum-free word expressible by the realized signature alphabet. In the binary MultiTag specialization, that threshold equals realized rank; the

substantive compiler work is proving that the corresponding deletion is sound and non-increasing. Only a separate compiler witness can turn the certificate ceiling into intrinsic support.

For MultiTag-TARE, block count controls deletion profitability through the $b - 1$ Restore penalty, while the realized alphabet controls sufficient support. The result is a strong normal form, not a necessity theorem.

References

1. N. Schillo, A. Sturm, and R. Quay, “TARE: Block Encoding Linear Combinations of Pauli Strings Without Ancilla State Preparation,” arXiv:2601.05740v4 [quant-ph] (2026).
2. I. Aliev, J. A. De Loera, F. Eisenbrand, T. Oertel, and R. Weismantel, “The Support of Integer Optimal Solutions,” *SIAM Journal on Optimization* **28**, 2152-2157 (2018). DOI: 10.1137/17M1162792
3. G. Wang, “The universal zero-sum invariant and weighted zero-sum for infinite abelian groups,” *Communications in Algebra* **53**(4), 1581-1599 (2025). DOI: 10.1080/00927872.2024.2418017